

# Introduzione all'UML

# Generalità

## Modellare

- Un modello è un'**astrazione** che cattura le proprietà salienti della **realtà** che si desidera rappresentare.
- Idealizza una realtà complessa, individuandone i tratti importanti e separandoli dai dettagli, facilitandone la comprensione.
- La mente umana compie un'attività continua di modellazione, producendo schemi per comprendere e spiegare quello che viene percepito dai sensi.
- La realtà è un'*istanza* del modello.

# Generalità

## Perché Modellare

- Per *comprendere* il soggetto in analisi.
- Per *conoscere* il soggetto in analisi (fissando ciò che si è compreso).
- Per *comunicare* la conoscenza del soggetto.

# Generalità

## I progetti software sono complessi

- Il tipico progetto software raramente coinvolge un solo sviluppatore, e può coinvolgerne anche centinaia:
  - ▶ separare compiti e responsabilità
  - ▶ raggruppare le informazioni a diversi livelli di granularità
- Il tipico progetto software subisce un ricambio di personale nel corso della sua storia:
  - ▶ il progetto perde conoscenza
  - ▶ nuovi sviluppatori devono acquisirla
- Le caratteristiche del progetto spesso mutano col tempo:
  - ▶ necessità di comunicare con il cliente in termini chiari
  - ▶ prevedere ed adattarsi ai cambiamenti
  - ▶ stimarne l'impatto su costi, tempi e risorse di sviluppo

Come si può ragionare su questo se non si sa su *cosa* si sta ragionando?

# Generalità

## I linguaggi di modellazione

- Un linguaggio di modellazione fornisce le primitive a cui ricondurre la realtà in esame.
- Permette di esprimere le **entità** che compongono un sistema complesso, le loro **caratteristiche** e le **relazioni** che le collegano.
- Nell'ambito di un progetto, il **linguaggio** di modellazione è normalmente distinto dal **processo** di sviluppo:
  - ▶ il linguaggio descrive cosa deve essere ottenuto;
  - ▶ il processo descrive i passi da intraprendere per ottenerlo;
  - ▶ insieme, linguaggio e processo definiscono un **metodo di sviluppo**.

# Generalità

## Che cos'è UML (1)

- **UML**, Unified Modeling Language, è un linguaggio **semiformale** e **grafico** (basato su diagrammi) per specificare, visualizzare, costruire e documentare gli artefatti di un sistema software.
- Permette di:
  - ▶ modellare un dominio
  - ▶ scrivere i requisiti di un sistema software
  - ▶ descrivere l'architettura del sistema
  - ▶ descrivere struttura e comportamento di un sistema
  - ▶ documentare un'applicazione
  - ▶ generare automaticamente un'implementazione
- Gli stessi modelli UML sono quindi artefatti usati per sviluppare il sistema e comunicare con il cliente (ma anche con progettisti/sviluppatori/etc.)

# Generalità

## Che cos'è UML (2)

- Si tratta di un linguaggio di modellazione usato per capire e descrivere le caratteristiche di un nuovo sistema o di uno esistente.
- Indipendente dall'ambito del progetto.
- Indipendente dal processo di sviluppo.
- Indipendente dal linguaggio di programmazione (progettato per essere abbinato alla maggior parte dei linguaggi object-oriented).
- Fa parte di un metodo di sviluppo, non è esso stesso il metodo.

# Generalità

## L come Linguaggio

- Si tratta di un vero e proprio linguaggio, non di una semplice notazione grafica.
- Un modello UML è costituito da un insieme di elementi che hanno *anche* una rappresentazione grafica.
- Il linguaggio è semiformale perché descritto in linguaggio naturale e con l'uso di diagrammi, cercando di ridurre al minimo le ambiguità.
- Ha regole *sintattiche* (come produrre modelli legali) e regole *semantiche* (come produrre modelli con un significato).



# Generalità

## Sintassi e semantica: esempio



- Usiamo un semplice diagramma dei Casi d'Uso come esempio.
- Regola sintattica: una relazione tra un attore e un caso d'uso può opzionalmente includere una freccia.
- Regola semantica: la freccia significa che la prima interazione si svolge nel senso indicato dalla freccia.

# Generalità

## U come Unificato

- UML rappresenta la sintesi di vari approcci metodologici fusi in un'unica entità.
- L'intento era di prendere il meglio da ciascuno dei diversi linguaggi esistenti e integrarli. Per questo motivo, UML è un linguaggio molto vasto.
- Questa è una delle critiche principali mosse a UML ("vuole fare troppe cose").

# Generalità

## Object-oriented

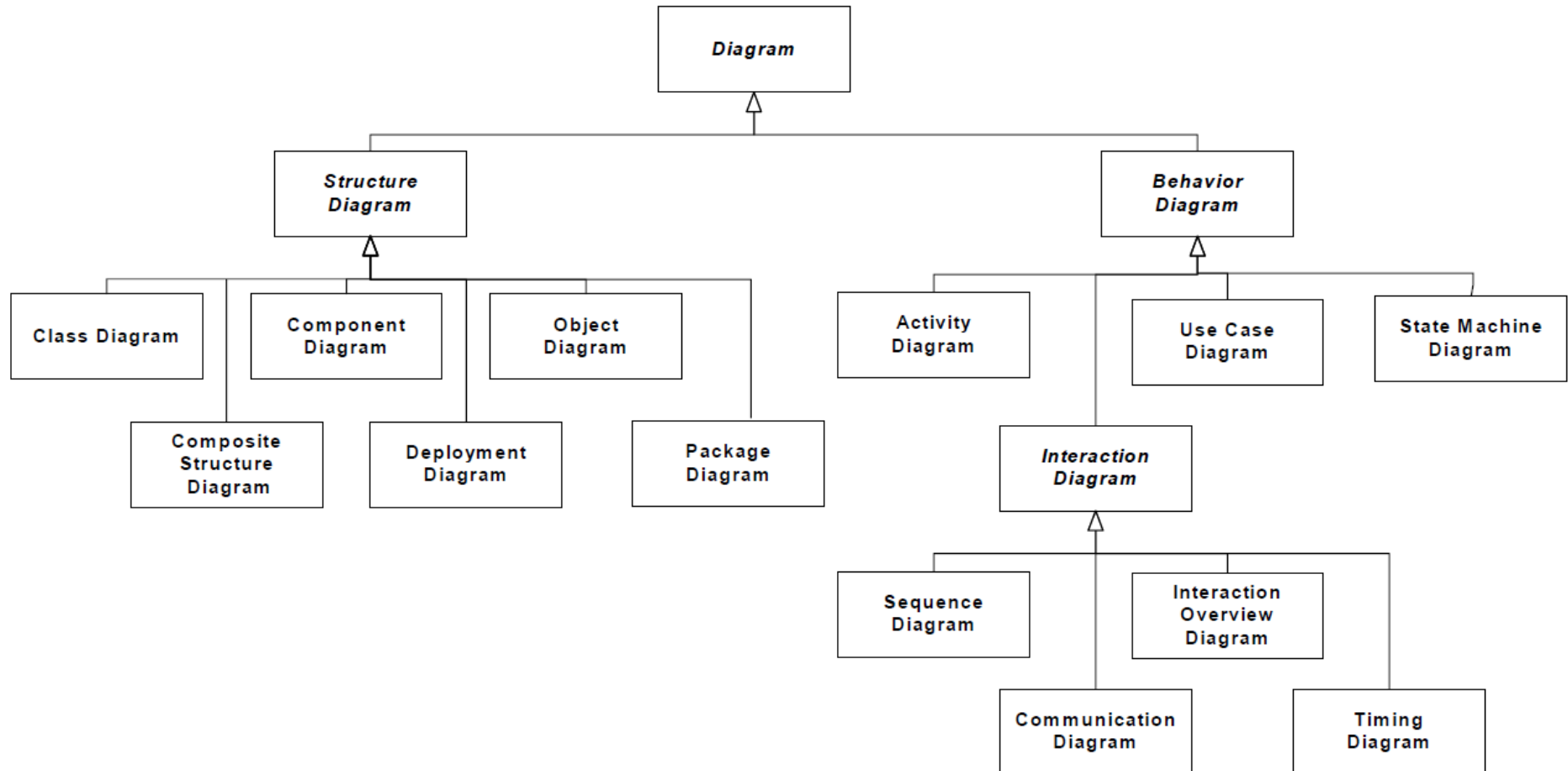
- UML non è solo un linguaggio per la modellazione, ma un linguaggio per la modellazione **orientata agli oggetti**.
- Questo include sia l'*analisi* che la *progettazione* orientata agli oggetti (OOA e OOD, rispettivamente):
  - ▶ **analisi**: capire **cosa** deve fare il sistema, senza occuparsi dei dettagli implementativi
  - ▶ **progettazione**: capire **come** il sistema raggiunge il suo scopo, come viene implementato
- UML offre strumenti di modellazione OO in entrambi gli ambiti; frammenti differenti di UML sono impiegati in diverse fasi del processo di sviluppo (anche se UML stesso non fornisce indicazioni sul suo utilizzo).

# Generalità

## I diagrammi e le viste

- Un **diagramma** è la **rappresentazione grafica di una parte del modello**.
- Fornisce una **vista** di un sistema o una sua parte, cioè ne mette in risalto diverse proprietà.

# Generalità



# Struttura

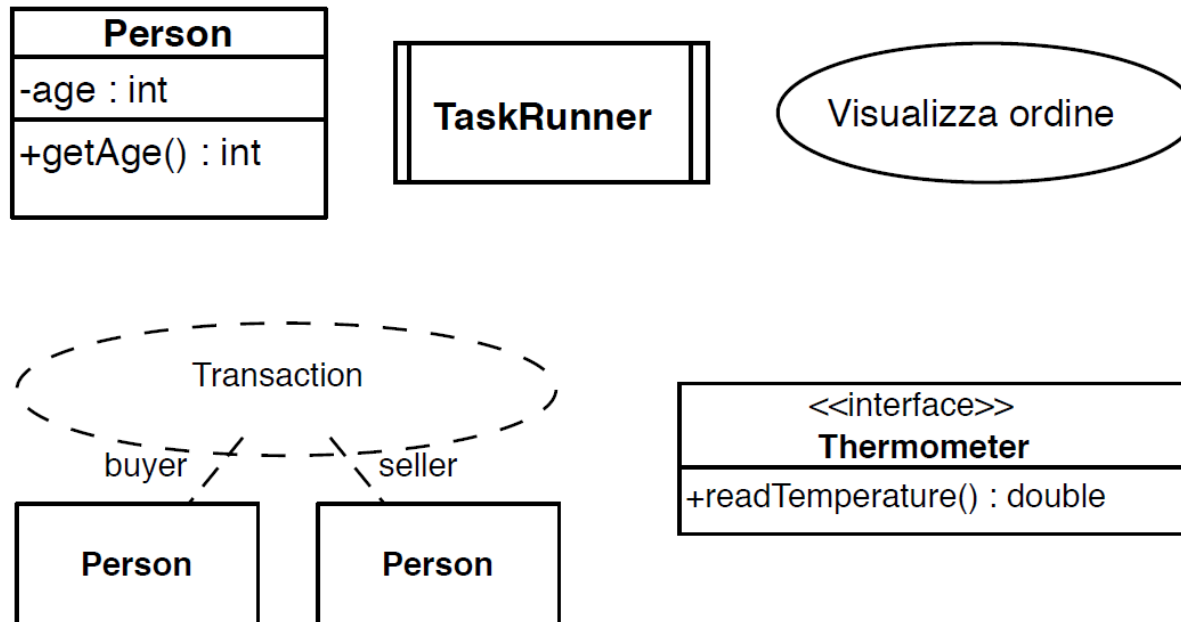
## Entità UML

- UML prevede diversi tipi di entità che possono essere organizzati in quattro categorie:
  - ▶ Strutturali
  - ▶ Comportamentali
  - ▶ Informative
  - ▶ Raggruppamento e contenimento

# Struttura

## UML: entità strutturali

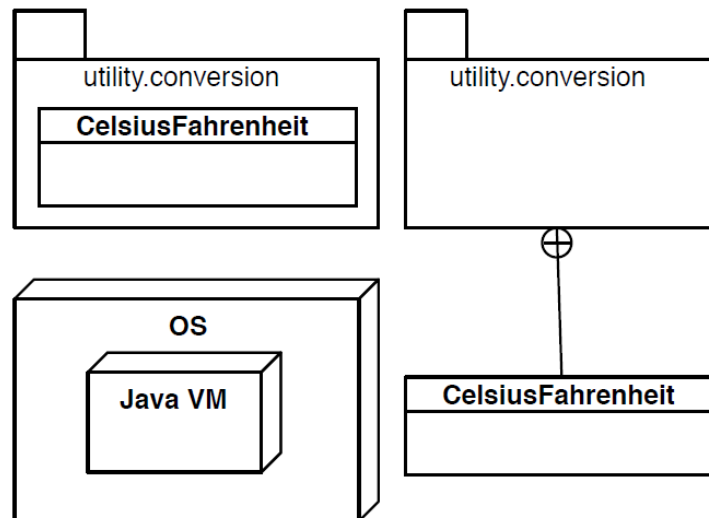
- Definiscono le "cose" (*classificatori, things*) del modello
- Alcuni esempi: classi, classi attive, use-case, collaborazioni, interfacce



# Struttura

## UML: entità di raggruppamento e contenimento

- I *package* raggruppano altri elementi e forniscono loro un *namespace* (che permette poi di identificare ogni elemento con il suo nome).
- In UML, moltissimi elementi possono contenere altri elementi al loro interno, formando una struttura gerarchica, rappresentabile graficamente in vari modi.

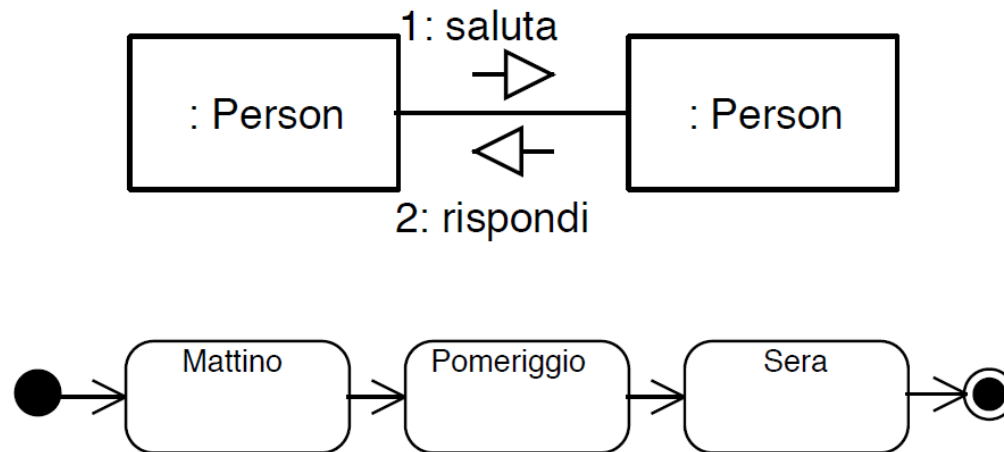




# Struttura

## UML: entità comportamentali

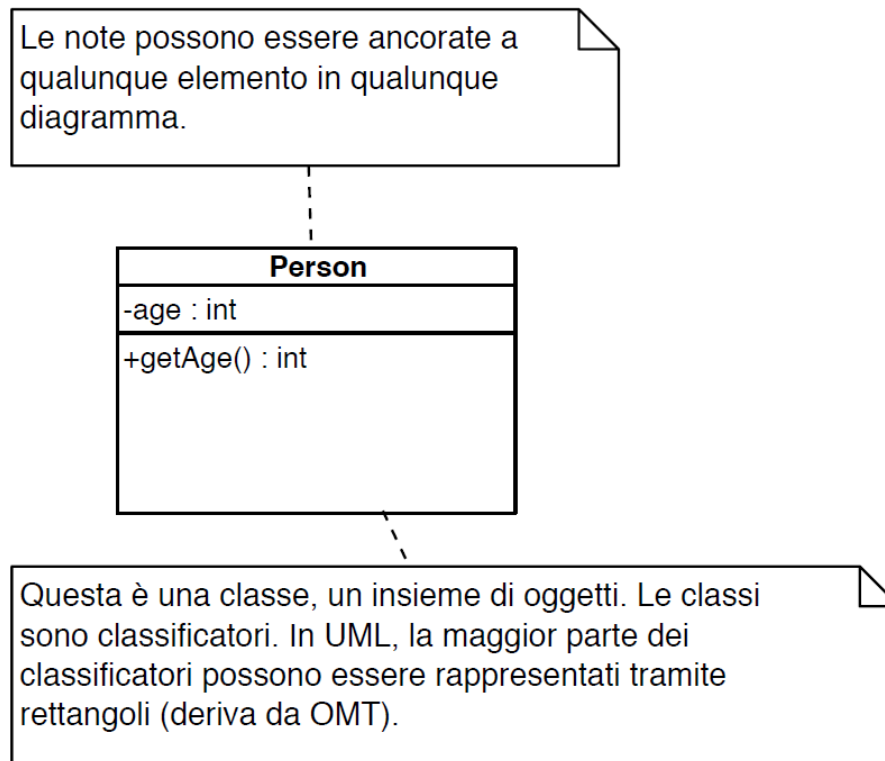
- Descrivono il "behavior": interazioni, collaborazioni (communication), scambi di messaggi, transizioni di stato, etc.



# Struttura

## Entità informative

- Uno degli scopi principali della modellazione è la leggibilità: un diagramma non leggibile e informativo serve a poco...
- Le *note* UML non hanno effetti sul modello ma migliorano la leggibilità.



# Struttura

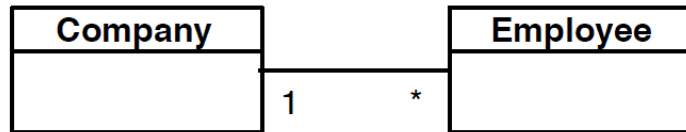
## Relazioni

- Gli elementi del modello possono essere collegati da **relazioni**.
- Rappresentate graficamente tramite linee.
- Possono avere un nome.
- Quattro sottotipi fondamentali:
  - ▶ **Association**
  - ▶ **Generalization**
  - ▶ **Dependency**
  - ▶ **Realization**

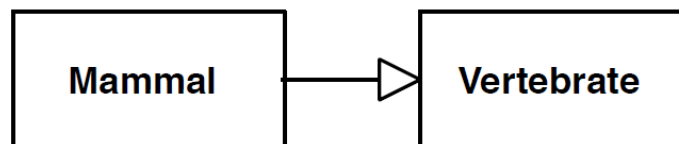
# Struttura

## Association e generalization

- Un'associazione descrive l'esistenza di un nesso tra le istanze di classificatori (*things*) e ha varie caratteristiche, alcune opzionali.



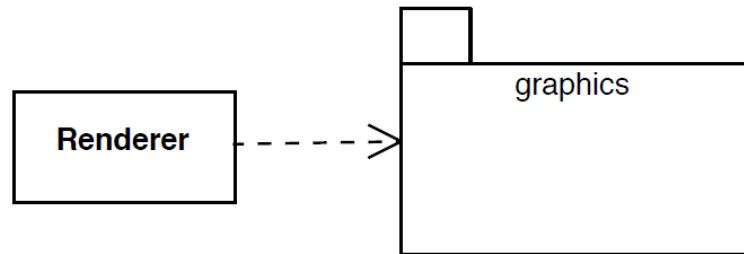
- La generalizzazione è una relazione tassonomica da un elemento specializzato verso un altro, più generale, dello stesso tipo.
  - ▶ Il figlio è sostituibile al genitore dovunque appaia, e ne condivide struttura e comportamento.



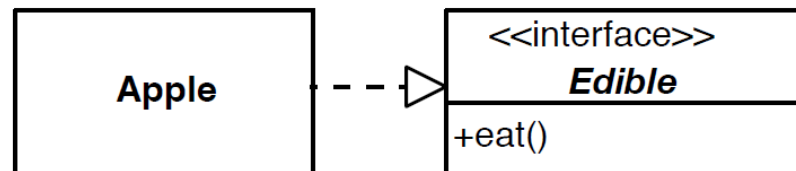
# Struttura

## Dipendenze e realizzazioni

- Una dipendenza è una relazione semantica: indica che il *client* dipende, semanticamente o strutturalmente, dal *supplier* (variazioni alla specifica del supplier possono cambiare quella del client).



- Anche la realizzazione è una relazione semantica: il supplier fornisce una specifica, il client la realizza (es: implementazione di interfacce, templates, etc.)



# Struttura

## Le frecce in UML

- Un metodo mnemonico per ricordarsi il verso di tutte le frecce in UML è il seguente:
- In UML, tutte le frecce vanno **da chi sa verso chi non sa** (dell'esistenza dell'altro).
- In una generalizzazione, il figlio sa di estendere il genitore, ma il genitore non sa di essere esteso.
- In una dipendenza, chi dipende sa da chi dipende, ma non vice-versa.
- In una realizzazione, chi implementa conosce la specifica, ma non il contrario.

# Struttura

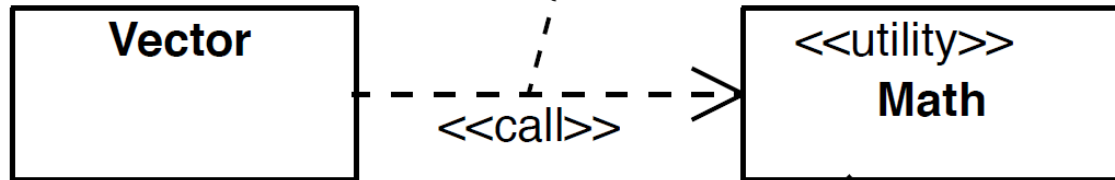
## Stereotipi

- Un'altra primitiva di UML comune ad ogni diagramma.
- Rende un diagramma più informativo arricchendo la semantica dei costrutti UML.
- Uno stereotipo è una parola chiave tra virgolette e abbinata ad un elemento del modello.
- Es. «import», «utility», «interface».

# Struttura

## Stereotipi in un diagramma delle classi

Questa dipendenza tra Vector e Math ha lo stereotipo «call»; significa che operazioni di Vector invocano operazioni di Math.



Lo stereotipo «utility» indica che questa è una classe utility, cioè una collezione di variabili globali e operazioni statiche usate da altre parti del sistema.

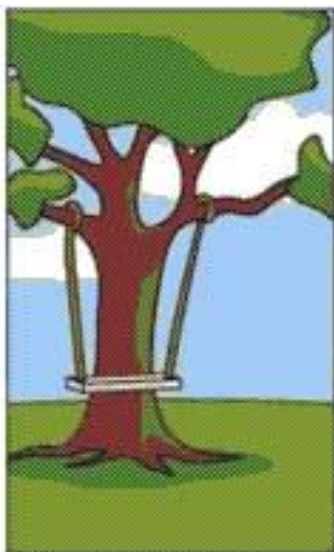


# Tools

- ArgoUML: <http://argouml.tigris.org/>
- Visual Paradigm: <https://www.visual-paradigm.com/solution/freeuml设计tool/>
- Papyrus: <http://eclipse.org/papyrus>
- Umbrello: <http://uml.sourceforge.net/>
- Eclipse: [www.eclipse.org/uml2/](http://www.eclipse.org/uml2/)



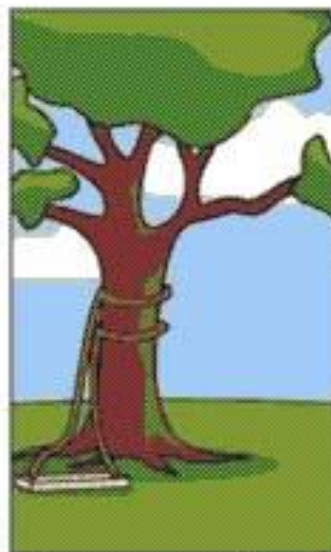
Cosa il cliente dichiara  
di volere



Cosa capì il capo  
progetto



Come lo progettò  
l'analista



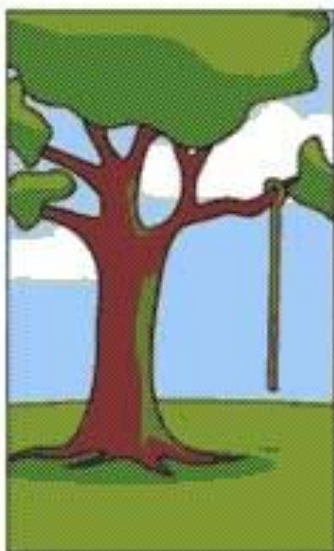
Cosa programmo  
il programmatore



Cosa definì il  
consulente



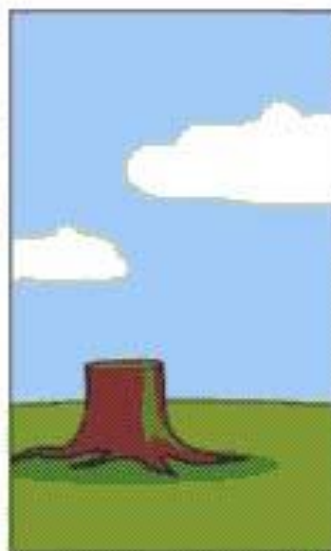
Come il progetto fu  
documentato



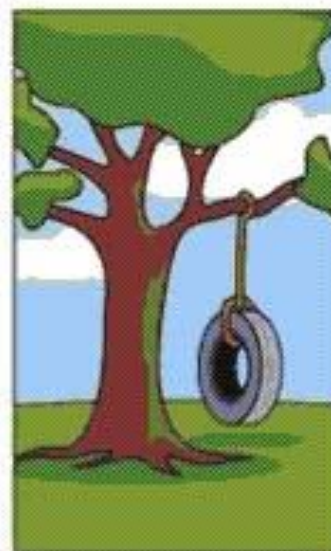
Cosa fu installato



Cosa fu messo in  
conto al cliente



Come fu eseguita  
la manutenzione



Quello di cui il cliente  
avrebbe avuto bisogno

# Analisi e specifica dei requisiti

## Obiettivi

---

- Introdurre i concetti di requisiti utente e di requisiti di sistema
- Descrivere i requisiti funzionali e non-funzionali
- Spiegare come i requisiti software possano essere organizzati in un documento di specifica dei requisiti

# Ingegneria dei requisiti

- Il processo condotto per stabilire i servizi che il cliente richiede al sistema, i vincoli operativi e quelli per lo sviluppo.
- I requisiti sono la descrizione di tali servizi e dei relativi vincoli.

# Che cos'è un requisito?

- Può variare tra una **descrizione astratta**, ad alto livello di qualcosa che il sistema farà, o di un suo vincolo, ed una **dettagliata specifica funzionale** matematica
- Ciò è inevitabile perchè un requisito svolge una duplice funzione:
  - Può essere la base per una gara fra potenziali sviluppatori: quindi deve essere aperto ad interpretazioni diverse
  - Può essere la base del contratto stesso- quindi deve essere definito in dettaglio
  - Entrambi si possono chiamare requisiti



# Necessità di diversi livelli di astrazione nei requisiti (Davis)

---

“Se una compagnia vuole dare in appalto un grande progetto di sviluppo software, deve definire le sue esigenze in modo abbastanza astratto, da non predefinire alcuna soluzione.

I requisiti devono essere scritti in modo da consentire a vari appaltatori di interpretare le esigenze e proporre vari metodi per soddisfarle.

Quando l'appalto è stato assegnato, l'appaltatore deve scrivere per il cliente una descrizione del sistema molto dettagliata, in modo che il cliente possa capire e validare ciò che il sistema farà.

Entrambi i documenti si chiameranno *documenti dei requisiti* del sistema.”

# Analisi e specifica dei requisiti

- **Analisi dei requisiti**
  - Processo di valutazione delle necessità del committente del software, con redazione di un documento di analisi dei requisiti, di solito in linguaggio naturale
  - Viene redatto al termine di una campagna di interviste svolte dagli analisti presso il committente
- **Specifica dei requisiti**
  - Processo di schematizzazione dei requisiti di un sistema, con redazione di un documento formattato rispetto ad uno standard
  - E' la base del contratto di fornitura

# Tipi di requisiti

- **Requisiti Utente**

- Frasi in linguaggio naturale (e diagrammi) relativi ai servizi che il sistema fornisce e i suoi vincoli operativi. Scritti per i clienti.

- **Requisiti di Sistema**

- Un documento strutturato che fornisce una descrizione dettagliata dei servizi del sistema. Può essere parte del contratto fra cliente e sviluppatore



# Definizioni e specifiche dei requisiti

- Es. di definizione di un requisito utente:
  - Il software deve fornire un mezzo per rappresentare ed accedere a file esterni creati da altri tools.
- Es. di Specifiche dei requisiti di sistema
  1. L'utente deve poter definire il tipo dei file esterni
  2. Ciascun tipo di file esterno deve avere uno strumento associato
  3. Ciascun tipo di file esterno deve essere rappresentato con una sua icona sul display utente
  4. Quando un utente clicca sull'icona di un file, l'effetto sarà di attivare lo strumento associato al tipo di quel file
  5. ...

# Requisiti funzionali e non funzionali

- Requisiti Funzionali
  - Frasi che descrivono ciò che il sistema dovrà fare, come reagirà agli input e si comporterà in varie situazioni.
- Requisiti Non-funzionali
  - Vincoli sui servizi offerti dal sistema (come vincoli sui tempi di risposta, ...) o a cui sottostare durante lo sviluppo → Requisiti di qualità
- Requisiti di dominio
  - Requisiti che derivano dal dominio di applicazione del sistema e ne riflettono le caratteristiche ed i limiti.

# Requisiti funzionali

- Descrivono le funzionalità e i servizi offerti.
- Anche in questo caso, due livelli di astrazione:
  - Requisiti funzionali utente: frasi ad alto livello su ciò che il sistema farà
  - Requisiti funzionali di sistema: descrizioni dettagliate dei servizi.

# Un esempio: Il sistema LIBSYS

- É un sistema che fornisce un'interfaccia unica per navigare in un insieme di database di articoli.
- Gli utenti possono cercare, scaricare, ordinare, stampare tali articoli per motivi di studio.

# Esempi di requisiti funzionali utente

1. L'utente dovrà essere in grado di cercare un documento o in tutto il database o in un sottoinsieme di esso.
2. Il sistema dovrà fornire all'utente appropriati visualizzatori per leggere tutti i documenti presenti nel database.
3. Ad ogni ordine di articolo deve essere assegnato un identificatore univoco (ORDER\_ID) che l'utente deve poter copiare nella propria area di salvataggio.

*Diversi livelli di dettaglio (es. 1 e 3)*

# Imprecisioni nei requisiti

- Requisiti imprecisi o ambigui possono essere interpretati in modi diversi da sviluppatori ed utenti.
- Es.: il termine ‘appropriati visualizzatori’ nel req. 2
  - L’utente intendeva avere specifici visualizzatori per ogni tipo di documento;
  - Lo sviluppatore ha interpretato che dovrà fornire solo un generico visualizzatore di testi.

# Requisiti Non-funzionali

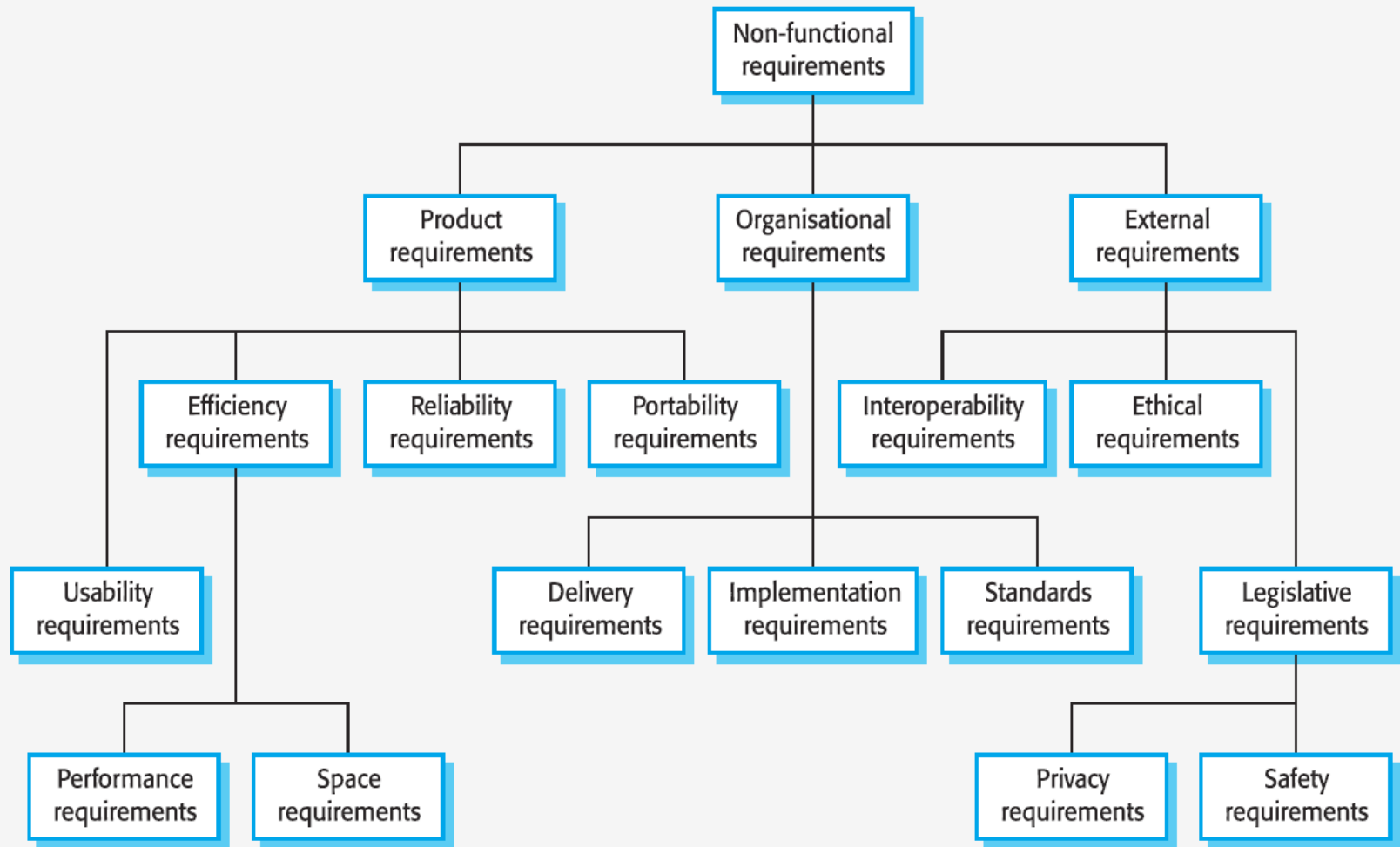
- Definiscono o limitano le proprietà del sistema (es. Affidabilità, tempi di risposta, memoria occupata...)
- Possono vincolare anche il processo di sviluppo da adottare (es. Uso di particolari standard per la documentazione, su sistemi CASE, linguaggi di codifica, metodi di sviluppo)
- Possono essere più critici dei req. funzionali: se non sono soddisfatti, il sistema è inutile.

# Tipi di requisiti Non-funzionali

- Requisiti del prodotto
  - Specificano il comportamento del prodotto (usabilità, efficienza, affidabilità, portabilità).
- Requisiti organizzativi
  - Derivano dalle politiche e procedure dell'organizzazione del cliente e dello sviluppatore (es. Standard di processo da usare, piattaforme, requisiti di consegna, etc.)
- Requisiti esterni
  - Derivano da fattori esterni al sistema e al suo processo di sviluppo (come i requisiti di interoperabilità, legislativi, etici, etc.)



# Tipi di requisiti Non-funzionali



# Problema del linguaggio naturale

- Mancanza di chiarezza
  - È difficile usare il linguaggio in modo preciso e non ambiguo, senza rendere il documento lungo e difficile da leggere.
- Confusione dei requisiti
  - Le varie tipologie di requisiti potrebbero non essere facilmente distinguibili.
- Mescolanza dei requisiti
  - Diversi requisiti potrebbero essere espressi insieme come un singolo requisito.

# Un requisito per una griglia su un editor grafico

---

**2.6 Funzionalità della griglia.** Per facilitare il posizionamento delle entità in un diagramma, l'utente può abilitare una griglia, in centimetri o pollici, attraverso una opzione del pannello di controllo. La griglia può essere abilitata e disabilitata durante le sessioni di lavoro, variando anche fra centimetri e pollici. Il requisito della griglia sarà presente anche quando si è scelta la visione ridotta dell'immagine (adattata alla finestra) ma, in tal caso, il numero di righe della griglia sarà ridotto.

# Problemi dei requisiti

---

- Il requisito della griglia combina tre tipi di requisiti:
  - Un requisito funzionale concettuale (il bisogno di una griglia);
  - Un requisito non-funzionale dettagliato (le unità di misura);
  - Un requisito non funzionale dell'Interfaccia Grafica definisce come la griglia può essere abilitata o disabilitata in vari casi.
- *I requisiti utente dovrebbero dare una descrizione generale del requisito, per essere compresi più facilmente e non confondere il lettore.*

# Esempio di descrizione

---

- *Nome requisito:* **Funzionalità della griglia**
- *Descrizione funzionalità:*
  - L'editor deve fornire una funzione griglia dove una matrice di linee orizzontali e verticali fornisce uno sfondo alla finestra dell'editor.
- *Motivazione:*
  - La griglia aiuta l'utente a costruire diagrammi più precisi
- *Specifica:*
  - ECLIPSE/WS/Tools/FS Sezione 5.6
- *Sorgente:*
  - Mario Rossi, Ufficio di Roma

# Esempio: Analisi dei requisiti per un Ristorante (1/4)

---

- Si vuole realizzare un sistema software che sia in grado di supportare la gestione di un ristorante. Esso deve consentire la gestione dell'anagrafica dei clienti, dando la possibilità di mantenere una scheda per ognuno di essi, in particolare per coloro che abbiano prenotato o organizzato cerimonie presso il ristorante.
- Dati anagrafici rilevanti sono considerati il Nome, il Cognome, la Data di Nascita, l'Indirizzo, la Città, il Codice Fiscale, i Numeri di telefono (eventuali) e gli indirizzi e-mail (eventuali).

# Esempio: Analisi dei requisiti per un Ristorante (2/4)

---

- Il sistema deve consentire l'inserimento, la visualizzazione, la modifica, la ricerca delle schede dei clienti. Inoltre, per ogni cliente, devono essere mantenuti dati statistici sulle operazioni da esso effettuate in passato, quali il numero di cerimonie effettuate, il numero medio annuo di cerimonie, l'incasso medio per cerimonia, l'incasso massimo, l'incasso minimo, il numero d'invitati medio, il numero di cerimonie disdette.
- Tali operazioni devono essere effettuate unicamente dal titolare, per cui sarà necessaria la realizzazione di un meccanismo per il controllo dell'identità dell'operatore, basato su di una login ed una password costanti.



# Esempio: Analisi dei requisiti per un Ristorante (3/4)

---

- Il sistema deve gestire le operazioni relative alla prenotazione del ristorante per cerimonie. La sala del ristorante può essere prenotata per una cerimonia solo da un cliente registrato, il quale deve indicare il tipo di cerimonia, la data e l'ora, il numero di invitati (distinguendo tra persone adulte e bambini), il numero di animatori, scegliere il menù (tra quelli prefissati oppure indicando un insieme di portate), definire tipologia e prezzo di eventuali servizi accessori e l'ammontare dell'eventuale acconto. La modifica dei dati della prenotazione è possibile solo per quello che riguarda il numero di invitati e il numero di animatori.
- E' possibile, inoltre, disdire la prenotazione della cerimonia. In tal caso, però, l'importo dell'eventuale acconto, deciso e pagato in sede di prenotazione della cerimonia, non viene restituito al cliente. Si noti che non è possibile prenotare cerimonie con meno di una settimana di preavviso e che in caso di prenotazione per un matrimonio, è necessario prenotare l'intera sala.



# Esempio: Analisi dei requisiti per un Ristorante (4/4)

---

- Il sistema deve poi gestire il pagamento ad avvenuta cerimonia lasciando libero il gestore di apportare arbitrariamente uno sconto sul prezzo inizialmente pattuito. Il sistema deve inoltre produrre una ricevuta fiscale o una fattura comprovante l'avvenuto pagamento.
- Il sistema deve presentare un'interfaccia utente basata su finestre e deve funzionare su calcolatori dotati di sistemi operativi Microsoft (Windows 98 o superiori). Inoltre si richiede che la memorizzazione dei dati sia effettuata su database Microsoft Access.

# In Documento di Specifica dei Requisiti (SRS)

- È una dichiarazione ufficiale di ciò che gli sviluppatori dovrebbero implementare (attenzione: il COSA ma non il COME)
- Deve includere sia i requisiti utente (RU), sia una specifica dettagliata dei requisiti di sistema (RS).
- Talora RU e RS sono contenuti nello stesso documento, talora in documenti separati.

# In Documento di Specifica dei Requisiti (SRS)

- Un SRS costituisce il punto di convergenza di tre diversi punti di vista: cliente, utente, sviluppatore
- Un SRS fornisce un punto di riferimento per la *validazione* del prodotto finale
- Un SRS di qualità è il pre-requisito per un software di alta qualità
  - un errore nell'SRS produrrà errori nel sistema finale
- Un SRS di qualità riduce i costi di sviluppo
  - correggere un errore dell'SRS dopo lo sviluppo costa 100 volte più che correggerlo durante la fase di Analisi

| FASE                     | COSTO (in ore uomo) |
|--------------------------|---------------------|
| Analisi dei Requisiti    | 2                   |
| Progettazione            | 5                   |
| Codifica                 | 15                  |
| Test di accettazione     | 50                  |
| Esercizio e manutenzione | 150                 |

# Lo standard IEEE 830 -1998

---

- Definisce una generica struttura per un documento dei requisiti che deve essere istanziata per ogni specifico sistema.  
Contiene più capitoli:
  - Introduzione.
  - Descrizione Generale.
  - Requisiti Specifici.
  - Appendici.
  - Indice.

# Standard *IEEE per l'SRS*

---

## Table of contents

### **1.Introduction**

- 1.1. Purpose (Scopo del documento)
- 1.2. Scope (Scopo del prodotto)
- 1.3. Definitions, Acronyms and Abbreviations
- 1.4. References
- 1.5. Overview (Descrizione generale del resto del documento)

### **2.General Description**

- 2.1. Product Perspective
- 2.2. Product Functions
- 2.3. User Characteristics
- 2.4. General Constraints
- 2.5. Assumptions and Dependencies

....

# Standard *IEEE per I'SRS*

## **3. Specific Requirements**

### 3.1. Functional Requirements

#### 3.1.1. Functional Requirement 1

3.1.1.1. Introduction

3.1.1.2. Inputs

3.1.1.3. Processing

3.1.1.4. Output

#### 3.1.2. Functional Requirement 2

...

#### 3.1.3. Functional Requirement n

...

# Esempio di Specifica di Funzionalità

---

Ciascuna Funzionalità deve essere specificata indicando gli input, l'output e l'azione elaborativa. Ad esempio:

## 3.2.1 Inserisci articolo

*Introduzione:* La funzionalità consente l'immissione dei dati relativi ad un nuovo articolo ed alla loro registrazione in un archivio

*Input:* codice articolo, descrizione, unità di misura

*Elaborazione:* visualizzazione di un form per l'immissione dei dati di input; lettura dei dati di input; verifica che l'unità di misura sia uno dei tre valori possibili e registrazione dei dati immessi nell'archivio Articoli

*Output:* dati di input registrati nell'archivio Articoli o messaggio di errore 'Unità di misura non ammessa'

# Altro esempio (1/3)

---

## **Elenco Fornitori**

- **Utenti della funzionalità:** Titolare
- **Introduzione**
  - La funzionalità visualizza la lista dei fornitori con i debiti verso questi e con possibilità di stampa. Consente, se richiesto, di selezionare un fornitore, di visualizzarne la scheda personale ed eventualmente di stamparla.



# Altro esempio (2/3)

---

- **Precondizione**

- L'utente si è precedentemente autenticato come titolare del ristorante

- **Input**

- Ragione sociale – Facoltativo
- Nome titolare – Facoltativo
- Cognome titolare – Facoltativo
- Indirizzo – Facoltativo
- Città – Facoltativo
- P. I.V.A. – Facoltativo
- Telefono – Un recapito telefonico. Facoltativo
- E-Mail – Un solo indirizzo. Facoltativo.
- Almeno uno tra le precedenti informazioni deve essere obbligatoriamente inserito.

# Altro esempio (3/3)

---

- **Elaborazione**

- Ristò presenta una lista con l'elenco dei fornitori
- Selezionando un fornitore dalla lista è possibile richiedere la visualizzazione o la stampa della sua scheda riportante tutti i dati anagrafici e contabili del fornitore.

- **Output**

- La lista con l'elenco dei fornitori, visualizzata a schermo o stampata, riporta per ciascuno di essi:
- Ragione Sociale - Nome Fornitore - Cognome Fornitore - Data d'inizio periodo Corrente - Saldo fornitore

# Le caratteristiche di un buon SRS ...

---

- Corretto/ Valido
  - se ogni requisito presente nell'SRS è realmente richiesto/necessario per gli utenti del sistema finale
- Completo
  - se ogni funzione richiesta al software ed il suo comportamento rispetto ad ogni possibile input è specificato
- Non Ambiguo
  - ogni requisito ha una sola interpretazione (formale vs. informale)
- Verificabile
  - se è possibile verificare che il sistema realizzi ogni requisito (richiede la non ambiguità dei requisiti)

# ... le caratteristiche di un buon SRS

---

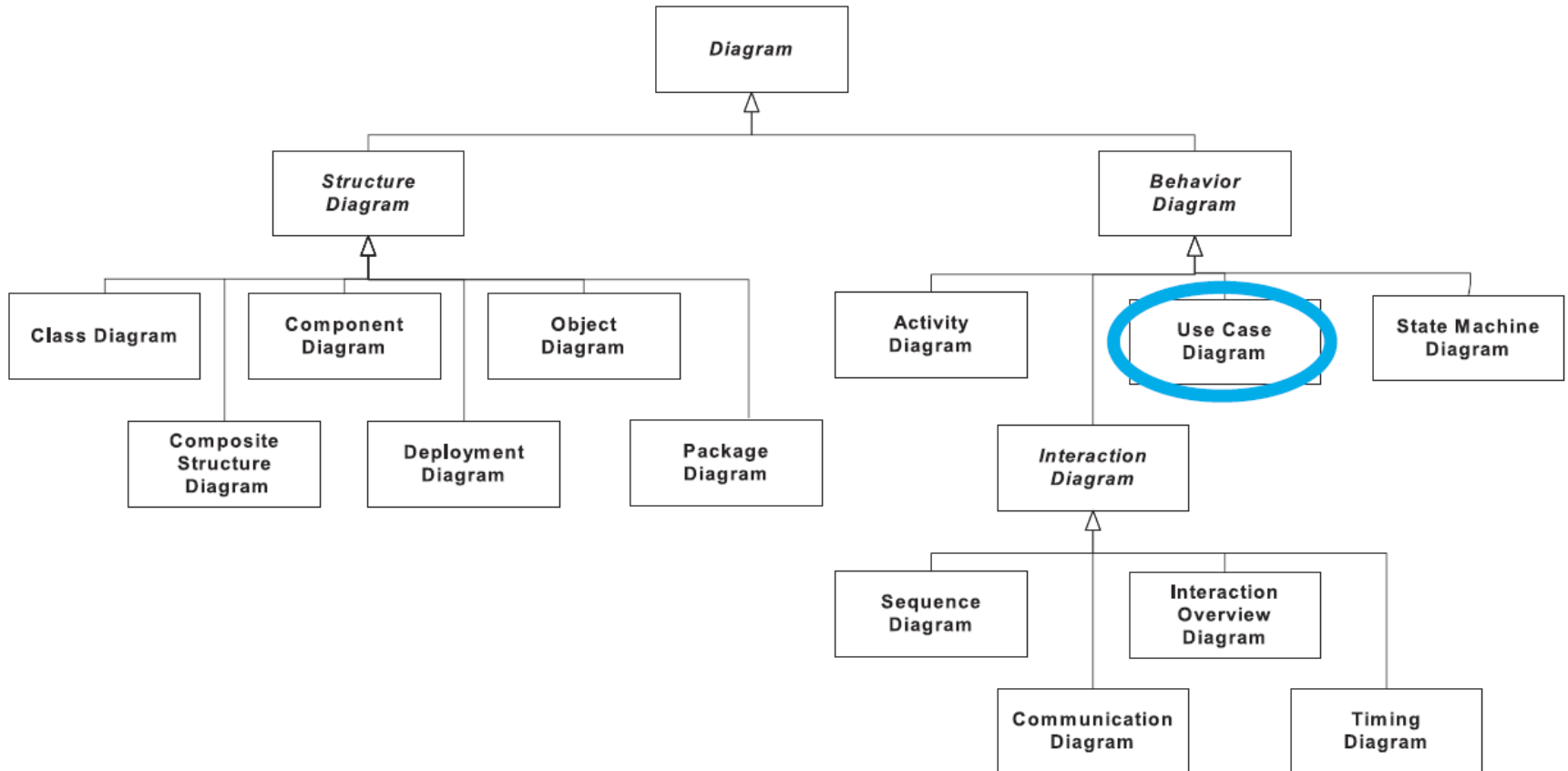
- Consistente
  - se nessun requisito è in contraddizione con gli altri
- Modificabile
  - se la struttura e lo stile dell'SRS sono tali da consentire facili modifiche, preservando consistenza e completezza (un SRS con ridondanze non si modifica facilmente)
- Tracciabile
  - se l'origine di ciascun requisito è chiara e può essere referenziata nello sviluppo futuro (def. IEEE)
  - *forward traceability*: requisito collegabile a qualche elemento del progetto e del codice
  - *backward traceability*: dal progetto e dal codice è possibile risalire al requisito corrispondente

# La Validazione delle Specifiche

---

- L'obiettivo è quello di assicurare che l'SRS rifletta accuratamente e con chiarezza i requisiti effettivamente richiesti al software
- Tipi di errori riscontrabili in un SRS
  - *Omissione* (mancata presenza di un requisito)
  - *Inconsistenza* (contraddizione fra i vari requisiti o dei requisiti rispetto all'ambiente operativo)
  - *Incorrettezza* (fatti non corretti nell'SRS)
  - *Ambiguità* (requisiti con significati multipli)

# Casi d'uso



# Caso d'uso: che cos'è e a cosa serve

- Si tratta di un diagramma che esprime un comportamento, offerto o desiderato, sulla base dei suoi risultati osservabili.
- L'oggetto esaminato è solitamente un sistema o una sua parte.
- Individua chi o che cosa ha a che fare con il sistema (*attore*) e che cosa viene fatto (*caso d'uso*).
- Si tratta tipicamente del primo tipo di diagramma ad essere creato in un processo o ciclo di sviluppo, nell'ambito dell'analisi dei requisiti.
- Modella i *requisiti funzionali* di un sistema.

# Requisiti funzionali (ripasso)

- I requisiti funzionali specificano cosa deve essere fatto.
- Sono indipendenti dalla tecnologia, dall'architettura, dalla piattaforma, dal linguaggio di programmazione.
- I requisiti non-funzionali specificano vincoli aggiuntivi, ad esempio:
  - ▶ performance
  - ▶ scalabilità
  - ▶ tolleranza ai guasti
  - ▶ dimensione degli eseguibili
- I casi d'uso sono particolarmente utili se la maggior parte dei requisiti sono funzionali.



# Attenzione!

- Questo diagramma specifica *cosa* va fatto, non *come* va fatto. Spetta alle fasi successive capire come realizzare i casi d'uso, la fase dei requisiti deve solo specificarli.
- Evitare riferimenti a tecnologie specifiche nei casi d'uso, per quanto possibile.
- Scegliere il giusto livello di dettaglio:
  - ▶ una granularità troppo fine rischia di sconfinare nella progettazione
  - ▶ una troppo grossa rischia di generare ambiguità nelle fasi successive
- Tenere in mente la leggibilità, questo diagramma dovrebbe risultare comprensibile anche a un non-esperto:
  - ▶ in particolare il cliente (è uno strumento per chiarirsi)

# I desiderata

- I *desiderata* sono ciò che il cliente desidera.
- Formalizzare i desiderata in requisiti è una delle più importanti sfide aperte dell'ingegneria del software.
- La specifica errata o incompleta delle richieste del cliente è una delle cause principali del fallimento dei progetti software.
- Il diagramma e la modellazione dei casi d'uso aiutano l'interazione con il cliente.

# I desiderata: esempio

Cliente:

- vorrei vendere i manufatti che realizzo...
- non vorrei solo un mercato locale...
- mi piacerebbe che gli acquirenti potessero visionare un catalogo da cui scegliere...
- vorrei gestire gli ordini da qualunque posto perché viaggio molto...

Che cosa viene fatto? Da chi?

# I desiderata: esempio

Cliente:

- vorrei avere la possibilità di creare un catalogo dei miei manufatti...
- vorrei un catalogo liberamente consultabile da chiunque...
- vorrei organizzare i manufatti raccogliendoli in categorie...
- vorrei che gli interessati all'acquisto potessero inviarmi un ordine, che io provvederò ad evadere previa una qualche forma di registrazione...

Desiderata riformulati in maniera migliore.

# Estrarre i requisiti

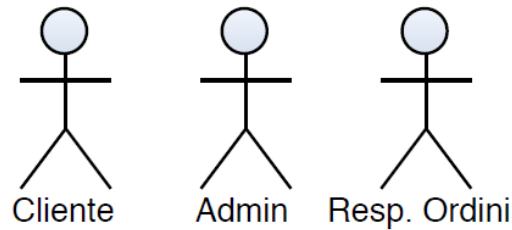
Chi interagisce con il sistema (attori)?

- Clienti
- Amministratori del negozio online
- Reparto ordini

Cosa fanno (casi d'uso)?

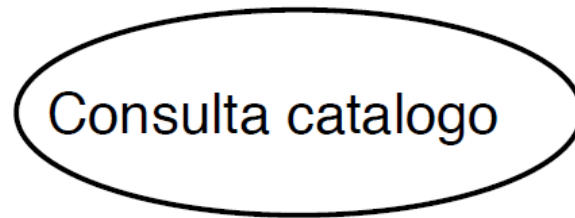
- Il cliente si registra, consulta il catalogo ed effettua acquisti
- L'amministratore organizza il catalogo, che è diviso in categorie
- Il reparto ordini riceve ordini da evadere

# Elementi del diagramma: attori



- Un attore specifica un ruolo assunto da un utente o altra entità che interagisce con l'argomento del diagramma nell'ambito di un'unità di funzionamento (caso d'uso).
- Un attore è esterno all'argomento (sistema) oggetto del diagramma.
- Non è necessariamente umano: oggetti fisici, agenti e componenti software, condizioni ambientali, ...
- Una singola entità o parte di entità può assumere molti ruoli, e una collezione di entità può essere rappresentata da un unico attore.

# Elementi del diagramma: casi d'uso



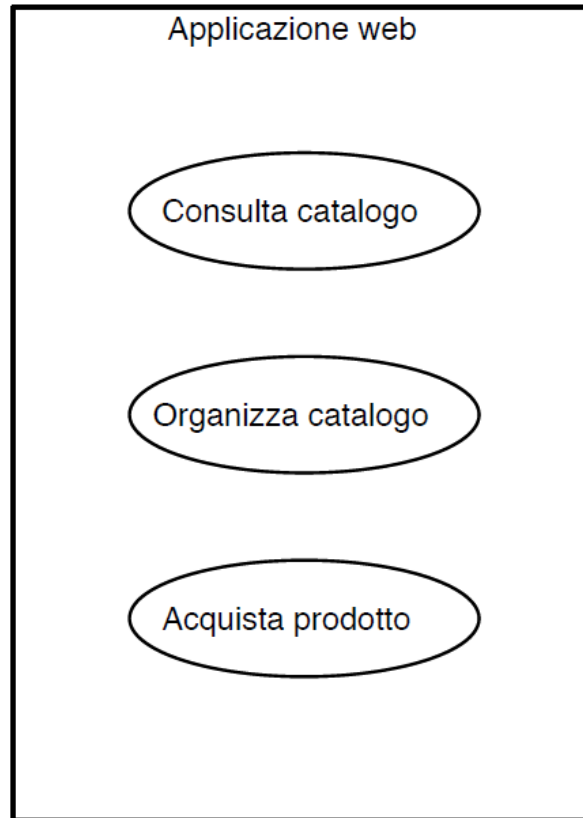
- Un caso d'uso specifica un insieme di azioni che producono un risultato osservabile per uno o più attori.
- Si tratta di un'unità coerente di funzionamento che il sistema fornisce ad uno o più utenti.
- La descrizione all'interno dovrebbe essere basata su un verbo o su un sostantivo esprimente un avvenimento.

# Elementi del diagramma: casi d'uso

- Un caso d'uso è sempre iniziato da un attore: in UML, un evento è sempre legato all'entità che lo ha generato.
- L'attore che inizia un caso d'uso è detto *primario*, gli altri attori che interagiscono nell'ambito di quel caso d'uso sono *secondari*.
- Un caso d'uso è un *classificatore dotato di comportamento*: può essere specificato da diagrammi di stato, interazione, sequenza, avere pre- e post-condizioni, ...
- Può ammettere al suo interno varianti al comportamento principale.

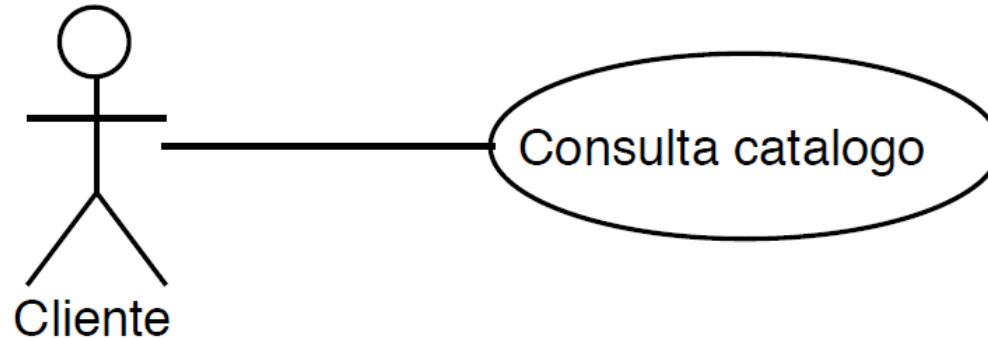


# Elementi del diagramma: sistema



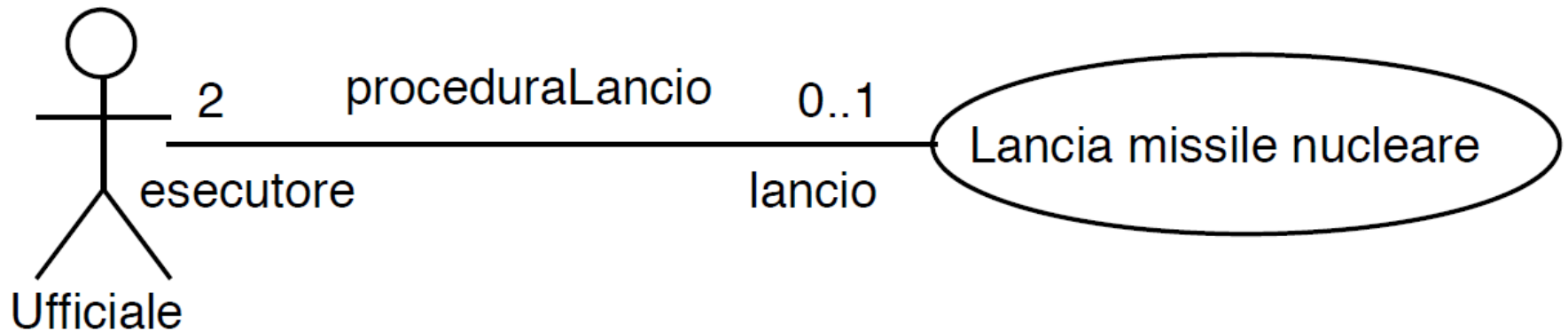
- Delimita l'argomento del diagramma, specificando i confini del sistema.

# Elementi del diagramma: associazioni



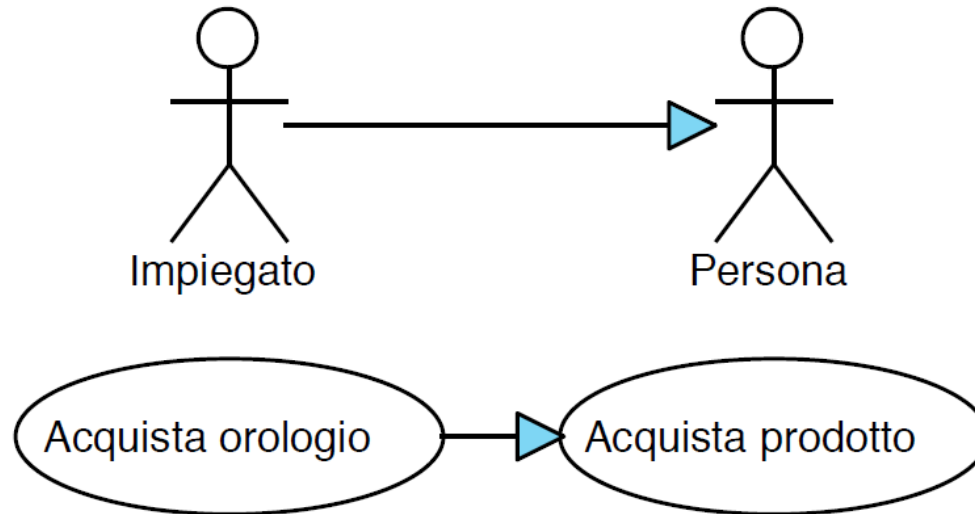
- Collega gli attori ai casi d'uso.
- Un attore si può associare solo a casi d'uso, classi e componenti.
- Un caso d'uso non si può associare ad altri casi d'uso riguardanti lo stesso argomento.
- Entrambi supportano solo associazioni binarie.

# Elementi del diagramma: associazioni



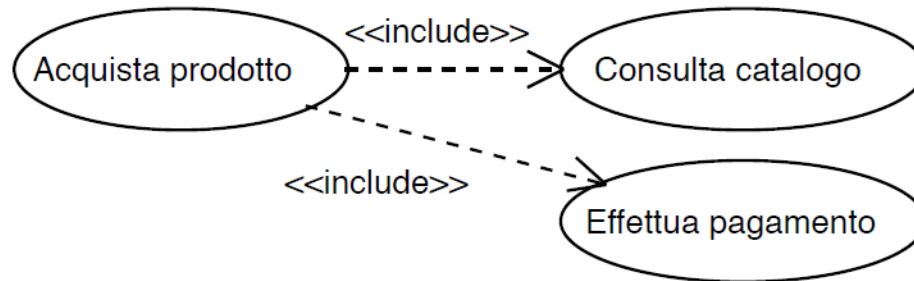
- Alcune caratteristiche opzionali comuni a tutte le associazioni in UML:
  - ▶ nome
  - ▶ molteplicità
  - ▶ ruoli

# Elementi del diagramma: generalizzazioni



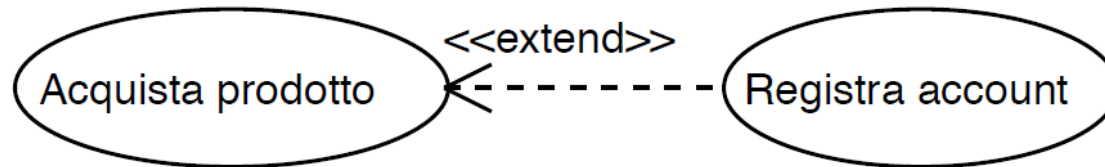
- Collega un attore o caso d'uso ad un altro più generale.
- Il figlio può sostituire il genitore dovunque questi appaia.
- In UML, elementi astratti (che non possono essere istanziati) hanno il nome in corsivo.

# Elementi del diagramma: include



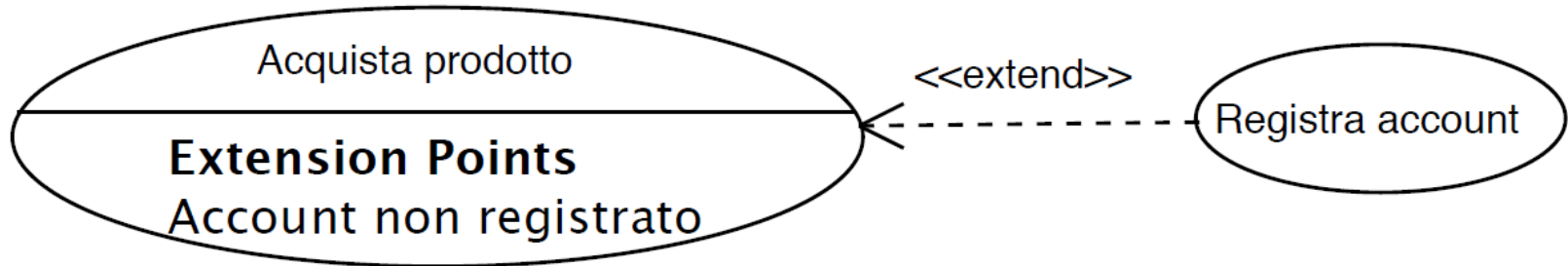
- Una dipendenza tra casi d'uso; il caso incluso fa parte del comportamento di quello che lo include.
- L'inclusione *non* è opzionale ed avviene in ogni istanza del caso d'uso.
- La corretta esecuzione del caso d'uso che include dipende da quella del caso d'uso incluso.
- Non si possono formare cicli di include.
- Usato anche per riutilizzare parti comuni a più casi d'uso.

# Elementi del diagramma: extend



- Una dipendenza tra casi d'uso (notare il verso della freccia).
- Il caso d'uso che estende (client) specifica un incremento di comportamento a quello esteso (supplier).
- Si tratta di comportamento *supplementare* ed *opzionale* che gestisce casi particolari o non standard.
- *Diverso* da una generalizzazione tra casi d'uso:
  - ▶ in una generalizzazione, entrambi i casi d'uso sono ugualmente significativi
  - ▶ in un extend, il client non ha necessariamente senso se preso da solo

# Extension points



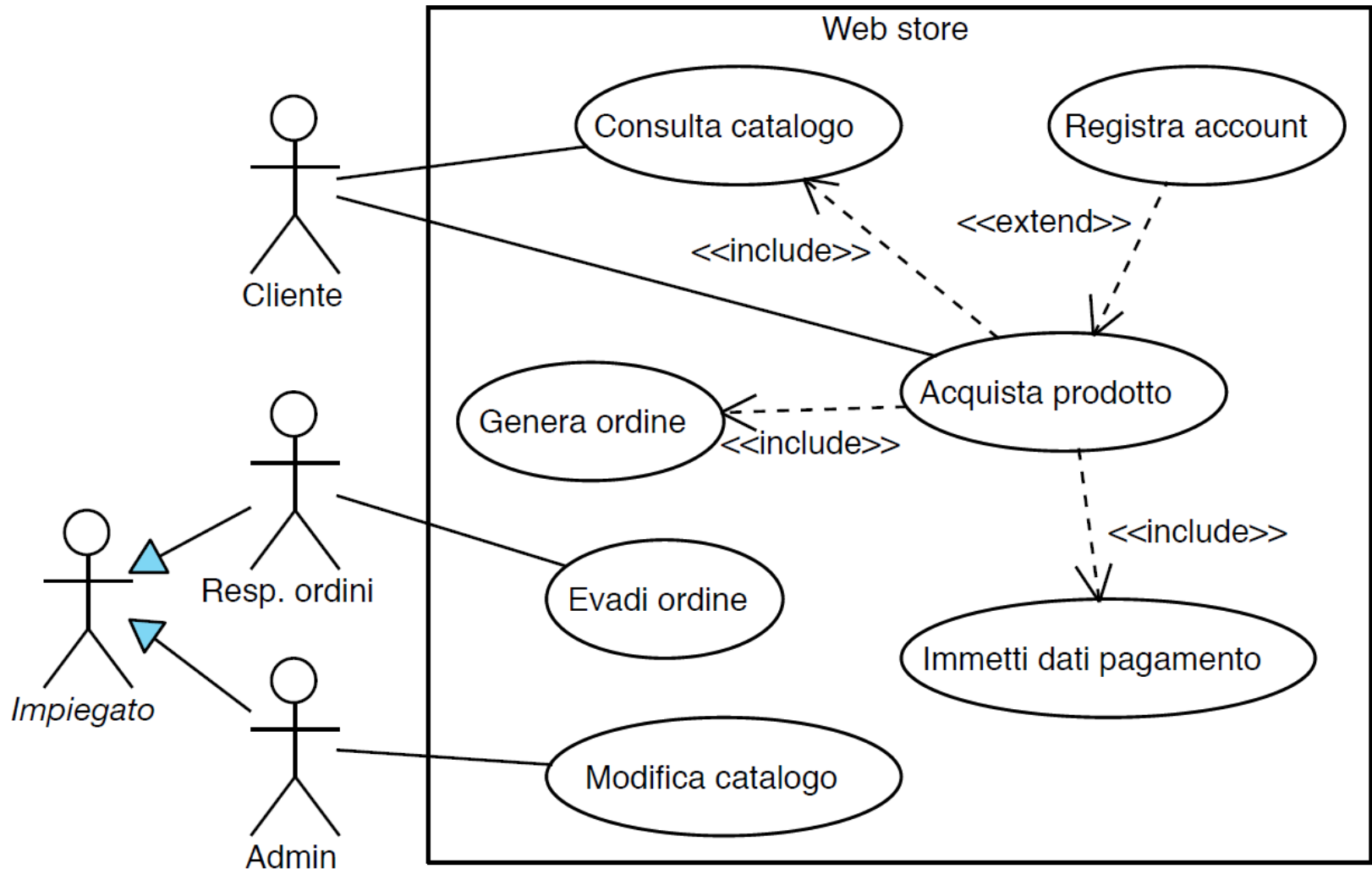
- Un caso d'uso raggiunto da almeno un extend può opzionalmente visualizzare i propri extension points.
- Specifica i punti e/o condizioni dell'esecuzione in cui il comportamento viene esteso.
- Se gli extension points sono molti, alcuni tool possono supportare la rappresentazione a rettangolo (i casi d'uso sono classificatori).

# Include VS Extend

- Include specifica comportamento *obbligatorio*.
- Extend specifica comportamento *supplementare*.
- Nell'include la freccia va dal caso d'uso che include verso quello incluso.
- Nell'extend la freccia va dal caso d'uso che estende verso quello esteso.
- Sono entrambi costrutti utili, ma non se ne deve abusare, o la leggibilità ne risente.



# Esempio



# Modellare i casi d'uso

- Se i requisiti del sistema non sono banali, nasce l'esigenza di abbinare i diagrammi dei casi d'uso a specifiche testuali più formali.
- I diagrammi dei casi d'uso non sono adatti a mostrare:
  - ▶ la sequenza temporale dei comportamenti
  - ▶ lo stato del sistema e degli attori prima e dopo l'esecuzione del caso d'uso
- Altri diagrammi (attività, stato, interazione) si occupano di queste viste, ma devono partire da una specifica.

# Specifica dei casi d'uso

- Ogni caso d'uso ha un nome e una specifica.
- La specifica è composta da:
  - ▶ *precondizioni (entry conditions)*: condizioni che devono essere vere prima che il caso d'uso possa essere eseguito (sono vincoli sullo stato iniziale del sistema)
  - ▶ *sequenza degli eventi (flow of events)*: i passi che compongono il caso d'uso
  - ▶ *postcondizioni (exit conditions)*: condizioni che devono essere vere quando il caso d'uso termina l'esecuzione

# Esempio di specifica dei casi d'uso

*Nome del caso d'uso*

*Identificatore univoco*

*Gli attori interessati dal caso d'uso*

*Lo stato del sistema prima che il caso d'uso possa iniziare*

*I passi del caso d'uso*

*Lo stato del sistema quando l'esecuzione del caso d'uso è terminata*

| Caso d'uso: Pagamento IVA                                                                                                                                                                                                      |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| ID: UC1                                                                                                                                                                                                                        |  |
| <b>Attori:</b><br>Tempo, Fisco                                                                                                                                                                                                 |  |
| <b>Precondizioni:</b><br>1. Si è concluso un trimestre fiscale                                                                                                                                                                 |  |
| <b>Sequenza degli eventi:</b><br>1. Il caso d'uso inizia quando si conclude un trimestre fiscale.<br>2. Il sistema calcola l'ammontare dell'IVA dovuta al Fisco.<br>3. Il sistema trasmette un pagamento elettronico al Fisco. |  |
| <b>Postcondizioni:</b><br>1. Il Fisco riceve l'importo IVA dovuto.                                                                                                                                                             |  |

# Sequenza degli eventi

- Un elenco di azioni che definisce il caso d'uso nella sua completezza.
- Il caso d'uso si considera eseguito solo se l'esecuzione arriva fino alla fine.
- Un'azione è sempre iniziata da un attore oppure dal sistema (in UML, gli eventi sono sempre legati a chi li crea).
- **Passo iniziale:** 1. Il caso d'uso inizia quando `<attore> <azione>...`
- **Passi successivi:** `<numero>.` Il `<attore/sistema>` `<azione>`

# Esempi

- ~~Incomincia quando si seleziona la funzione 'ordina libro'~~
- Il caso d'uso inizia quando il cliente seleziona la funzione 'ordina libro'
- ~~Vengono inseriti i dati del cliente~~
- Il cliente inserisce nel form il suo nome e indirizzo
- Il sistema verifica i dati del Cliente

# Esempio: negozio online

- Si consideri un negozio che rende disponibile un catalogo liberamente consultabile on-line. Gli utenti registrati possono inviare un ordine di acquisto (comunicando i dati di pagamento), che viene memorizzato nel sistema e trasferito al reparto ordini che lo evade.
- Si rappresenti il sistema con un diagramma dei casi d'uso.

Esempio: negozio online

**Chi** interagisce con il sistema (**attori**)?

**Cosa** fanno (**casi d'uso**)?



# Esempio: negozio online

**Chi** interagisce con il sistema (**attori**)?

- Clienti
- Amministratori del negozio online
- Reparto ordini

**Cosa** fanno (**casi d'uso**)?

# Esempio: negozio online

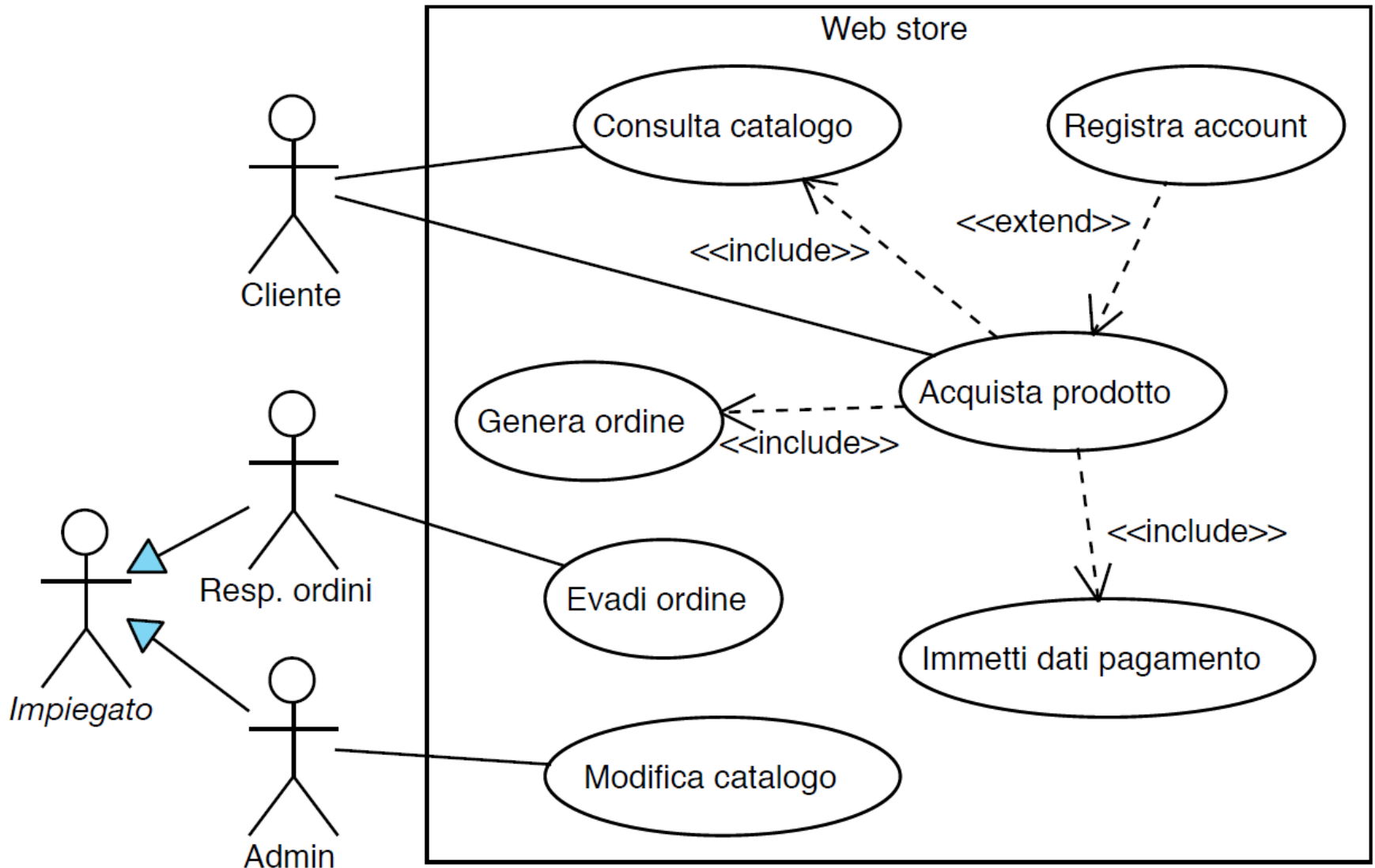
**Chi** interagisce con il sistema (**attori**)?

- Clienti
- Amministratori del negozio online
- Reparto ordini

**Cosa** fanno (**casi d'uso**)?

- Il cliente si registra, consulta il catalogo ed effettua acquisti
- Il cliente sceglie il tipo di pagamento
- L'amministratore organizza il catalogo
- Il reparto ordini evade gli ordini

# Esempio: negozio online



# Riferimenti

- Introduzione a UML – Angelo Di Iorio
- Analisi e specifica dei requisiti - Porfirio Tramontana
- Ian Sommerville, Ingegneria del Software, capitoli 6,7